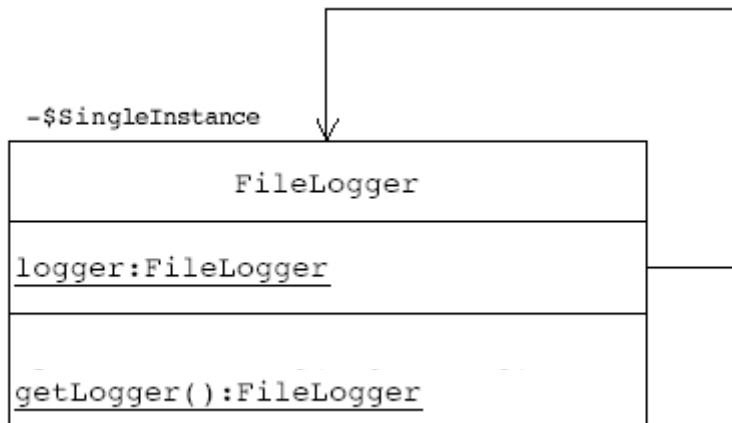


Corrigé Série de td1

Exercice 1

Le terme adapter est utilisé dans l'api Java pour désigner une classe utilitaire fournissant une implémentation vide pour toutes les méthodes d'une interface donnée. Ces classes adaptateurs sont utiles lorsque l'on voudrait implémenter une interface et que seulement quelques méthodes de cette interface nous intéressent. Au lieu d'implémenter l'interface et écrire un lot de méthodes à corps vide ; vaut mieux étendre l'adaptateur de l'interface et redéfinir uniquement les méthodes qui nous intéressent.

Exercice 2



```
public class FileLogger implements Logger{
```

```
private static FileLogger logger ;
```

```
private FileLogger(){}
```

```
public static FileLogger getFileLogger(){
    if (logger==null) logger = new FileLogger() ;
    return logger ;}
}
```

Exercice 3

```
import java.util.*;
public class AdapterExample {
    public static void main(String[] args) {
        LinkedList container = new LinkedList();
        container.add(new Integer(1));
        container.add(new Integer(2));
        Iterator iterator = container.iterator();

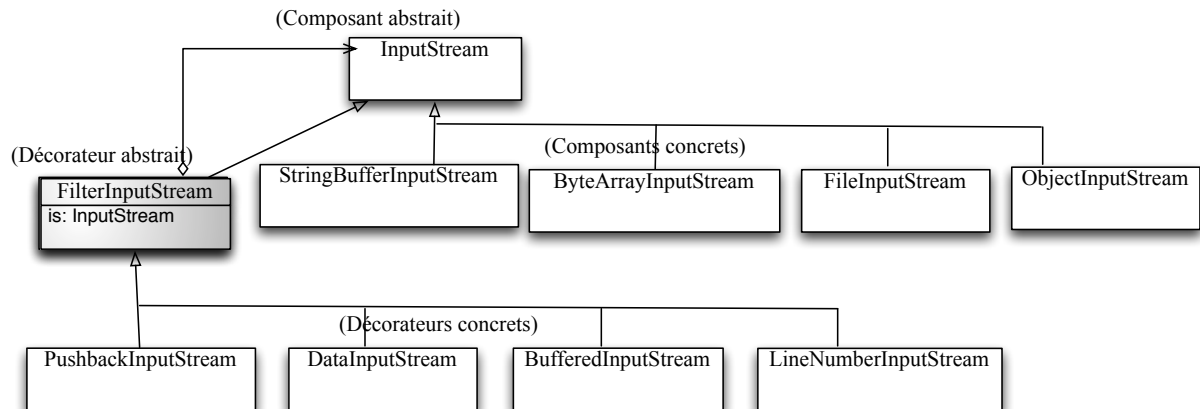
        IterationAdapter iterationAdapter = new IterationAdapter(iterator);
        client(iterationAdapter);
    }
    // ce client comprend
    //seulement l'ancienne interface Enumeration
    public static void client(Enumeration e) {
        while (e.hasMoreElements())
        { System.out.println(e.nextElement());
        } }
}
// Implémenter cette classe:
class IterationAdapter implements Enumeration
```

```

Iterator iter;
public IterationAdapter(Iterator iter)
{this.iter=iter;}
Boolean hasMoreElements(){
return iter.hasNext(); }
Boolean nextElement(){return iter.next();}}

```

Exercise 4



Exercise 5

Solution

```

public interface ComposantVisuel{
public void draw() ;}

```

```

public class TextView implements
ComposantVisuel{
public void draw(){//appel de la méthode qui va
//créer la vue éditeur de texte}}

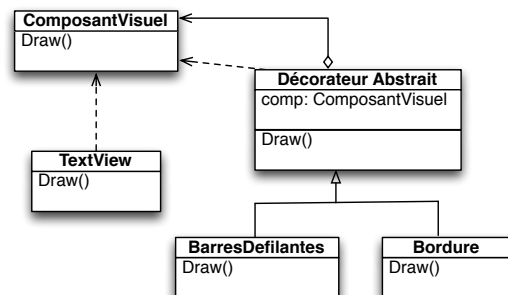
```

```

public class DécorateurAbstrait implements
ComposantVisuel{
ComposantVisuel comp ;
public DécorateurAbstrait (ComposantVisuel
comp){
this.comp=comp ;
}
public void draw(){comp.draw() ;} ;
}
public class BarresDefilantes extends DécorateurAbstrait{
public BarresDefilantes(ComposantVisuel comp){
super(comp) ;
}
public void draw(){
super.draw() ;
//appel à la méthode de dessin des barresDefilantes ;

}
}

```



```

public class Bordure extends DécorateurAbstrait{
public Bordure (ComposantVisuel comp){
super(comp) ;
}
public void draw(){
super.draw() ;
//appel à la méthode de dessin des bordures ;

}
}

```

Exercice 6

```

FileReader file = new FileReader();
// Comprimez le fichier
ZipReader zip = new ZipReader(file);
// Crypter le fichier zip
EncryptedReader enc = new
EncryptedReader(zip);
enc.Read();

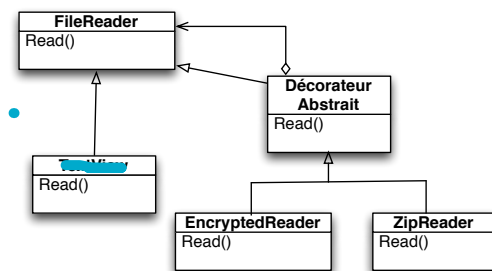
```

Autre solution:

```

EncryptedReader enc=new
EncryptedReader(new ZipReader(new
FileReader()));
enc.Read();

```



2-

```

EncryptedReader cyp=new EncryptedReader(new ZipReader(new
EncryptedReader (new FileReader())));
cyp.Read();

```

Exercice 7

```

BufferedReader input=new BufferedReader (new InputStreamReader(cs.getInputStream()));
PrintWriter out=new PrintWriter(sa.getOutputStream(),true);

```